

Multi-Core Global Scheduling —

Milestone 1: Technical Understanding and Research Framing

Md Mostafizur Rahman

Electronic Engineering, Hochschule Hamm-Lippstadt (HSHL)
Lippstadt, Germany md-mostafizur.rahman@stud.hshl.de

Abstract—Structured technical briefing (M1). Seed reference: G. Gracioli, A. A. Fröhlich, R. Pellizzoni, S. Fischmeister, “Implementation and Evaluation of Global and Partitioned Scheduling in a Real-Time OS,” *Real-Time Systems* 49:669–714, 2013. This milestone documents technical understanding of the paper before contextualization (M2) and critical evaluation/transfer (M3). Notes are deliberately structured, not publication prose.

I. PROBLEM STATEMENT AND MOTIVATION

- Multicore chips now consolidate functions previously split across networks of single-core ECUs (automotive ADAS: emergency braking, sensor fusion, night-vision / machine-vision pipelines). Motivation: cut wiring, power, space, cost.
- In hard real-time (HRT) systems correctness = *value* + *timing*: a correct result delivered after its deadline is a failure. A multicore HRT design is therefore a *scheduling policy* plus an *implementation with bounded timing cost*.
- Two dominant multicore paradigms:
 - **G-EDF** — one global deadline-ordered queue; the m earliest-deadline jobs run; migration allowed.
 - **P-EDF** — tasks bound to cores offline; per-core EDF; no migration.
- **Gap addressed:** every prior overhead comparison of these policies was run on Linux real-time patches (LITMUS^{RT}, SCHED_DEADLINE, ...). Linux internals (allocator, VFS, IRQ path) inject non-deterministic spikes that belong to the *OS substrate*, not the scheduler, and obscure the true policy overhead.
- No prior work measured G-EDF/P-EDF overhead on an RTOS *designed from scratch* for HRT $\Rightarrow$ the real distance between theoretical and practical schedulability was unknown.
- **Research question (paper):** how strongly does OS *design quality* affect schedulability, and can a purpose-built RTOS close the gap to theoretical bounds?
- **Research framing (my report):** if the OS substrate can dominate the algorithm, the effect should be (i) observable even on commodity general-purpose OSes and (ii) formally checkable as a deadline consequence. This motivates my transfer experiment (Windows/Linux WCET) and a UP-PAAL timing model, developed in M3.

II. CORE TECHNICAL IDEA (MAIN CONTRIBUTION)

Three bundled contributions:

- **EPOS + G-EDF.** G-EDF added to the purpose-built RTOS EPOS through compile-time metaprogramming: the global/partitioned policy is a C++ trait resolved at compile time, so policy selection carries *zero runtime branch cost*. Claimed first open-source RTOS supporting global multi-core real-time scheduling.
- **Empirical overhead measurement.** Five OS overhead sources (context switch, scheduling decision, thread release, timer tick, IPI latency) measured on EPOS vs LITMUS^{RT} on identical hardware (Intel i7-2600, 4 cores / 8 logical).
- **Schedulability comparison.** G-EDF vs P-EDF schedulability ratios, *with and without* measured overhead, over 18 synthetic task-set families (3 period $\times$ 3 utilisation $\times$ 2 shapes) and up to 100 processors.

Central claim. OS quality is a first-class variable in schedulability. It can *reverse* the theoretical algorithm ordering: in light-utilisation, short-period regimes, low-overhead G-EDF on EPOS can outperform P-EDF on LITMUS^{RT}, even though idealized theory tends to favour partitioning.

III. FORMALISM, MODEL, AND ARCHITECTURE

A. Periodic task model

Task set $\tau = \{T_1, \dots, T_n\}$ on m identical cores $\{P_1, \dots, P_m\}$. Each $T_i = (e_i, p_i, d_i)$ with WCET e_i , period p_i , deadline d_i . Implicit-deadline model $d_i = p_i$, and

$$u_i = \frac{e_i}{p_i}, \quad U = \sum_{i=1}^n u_i. \quad (1)$$

The j -th job of T_i released at r_i^j has absolute deadline $a_i^j = r_i^j + d_i$; the set is HRT-schedulable iff every job finishes by a_i^j . For uniprocessor EDF with implicit deadlines, $U \leq 1$ is *necessary and sufficient* (Liu & Layland, 1973). Multiprocessor G-EDF relies instead on *sufficient* tests that may reject feasible sets.

B. Scheduling algorithms

G-EDF keeps one global queue ordered by a_i^j . On release, if the new job's deadline precedes that of the latest-deadline running job, that job is preempted and an IPI reschedules the remote core; insertion is $O(n)$, dispatch $O(1)$. Migration may evict a task's working set from the previous core's cache (CPMD). Core loop:

- 1) on release of J_i at t : $J_i.\text{dl} \leftarrow t + d_i$; insert into Q $\{O(n)\}$
- 2) $J_{\text{low}} \leftarrow$ running job with latest deadline

- 3) if $J_{i,dl} < J_{low,dl}$: preempt J_{low} ; reinsert; IPI its core
- 4) on core free: dispatch $Q.pop_head()$ $\{O(1)\}$

P-EDF splits scheduling into an offline bin-packing phase (FFD/BFD/WFD) plus online per-core EDF, verified by

$$\sum_{T_i \in P_k} u_i \leq 1, \quad k = 1, \dots, m. \quad (2)$$

Bin-packing is NP-hard; heavy tasks ($u_i > 0.5$) can block packing entirely — e.g. five tasks with $u_i = 0.51$ do not fit on four cores although $U = 2.55 < 4$. No migration $\Rightarrow$ no IPI, no migration CPMD.

C. Preemption-centric WCET inflation

Implementation cost is folded into the analysed execution time before applying schedulability tests. With tick period Q and per-source costs $c^{xs}, c^{scd}, c^{rel}, c^{tck}, c^{ipi}, c^{pd}$:

$$u_0^{tck} = \frac{c^{tck} + c^{rel}}{Q}, \quad c^{pre} = \frac{c^{tck} + c^{rel}}{1 - u_0^{tck}}, \quad (3)$$

$$e'_i = \frac{e_i + 2(c^{scd} + c^{xs}) + c^{pd}}{1 - u_0^{tck}} + 2c^{pre} + c^{ipi}. \quad (4)$$

IPI latency is charged as *extra computation* because the G-EDF tests used cannot model self-suspension.

D. EPOS architecture (why overhead is low)

- **Compile-time criterion.** Policy (EDF, G-EDF, CPU-affinity/P-EDF) encoded as a `Criterion` class; the `GLOBAL_SCHEDULER` flag is resolved at compile time — no runtime policy branch.
- **Specialised queue.** `Scheduling_Queue` compiled to a single global ordered list (G-EDF) or per-core lists (P-EDF).
- **Inlined mediators.** Hardware mediators (IPI send, context switch, timer) are diluted into the application at compile time — no HAL, no system-call layer.
- **Library mode.** OS linked directly with the application: no kernel/user boundary, no MMU cost — but no memory protection.
- **Distributed alarm handler.** Each core owns its event list; all events reaching zero ticks are released in the same interrupt — no tick-delay on release, guaranteeing the m highest-priority threads run.

E. Weighted schedulability metric

To compress CPMD-dependent results into a 2-D plot,

$$W(D_c) = \frac{\sum_{U \in Q} U S(U, D_c)}{\sum_{U \in Q} U}, \quad (5)$$

where $S(U, D_c)$ is the schedulability ratio at utilisation cap U and CPMD bound D_c ; weighting by U prevents light and heavy loads being treated as equally informative. D_c is fed from measured worst-case CPMD (HW performance counters + `wbinvd`-forced eviction).

TABLE I
CPMD VS. WORKING-SET SIZE (WSS), 17-2600, 8 MB L3

WSS	Worst-case CPMD	Dominant effect
4 KB	$\sim 1 \mu s$	L1/L2 hit; prefetcher helps
128 KB	few μs	L2 boundary
512 KB	rising	L3 contention
10 MB	$> 100 \mu s$	LLC thrash; P-EDF $\approx$ G-EDF

IV. ASSUMPTIONS AND SCOPE LIMITATIONS

- **Task model.** Periodic, implicit deadlines only; no sporadic/apperiodic/constrained-deadline/mixed-criticality tasks. G-EDF's soft-real-time tardiness bound is mentioned but not measured.
- **Test type.** All 8 G-EDF tests + P-EDF heuristics are *sufficient, not necessary*: a set failing every test may still be feasible. Pessimism is worst for heavy/bimodal G-EDF.
- **Workload.** Synthetic CPU-bound loop (fixed arithmetic iterations); no I/O, no shared data, tiny cache footprint. Memory- or I/O-bound tasks would show different overhead.
- **Core count.** Overhead measured only on 8 logical cores; the 100-processor comparison uses *idealized, overhead-free* tests on a cluster (~ 1.1 CPU-years). No overhead data exist for > 8 cores.
- **Deployment.** EPOS library mode has no isolation $\Rightarrow$ unsuitable for safety certification (ISO 26262 ASIL-D).
- **Outlier handling.** Individual outliers (context-switch spikes up to 2 ms) are removed rather than a standard 1% tail — yet for HRT those tails are exactly the deadline misses.
- **Platform.** Single ccNUMA Sandy-Bridge node with QPI; results may not transfer to ARM Cortex-A, heterogeneous cores, or distributed NUMA.

V. RUNTIME, MEMORY, AND SCALABILITY ASPECTS

- **Queue cost.** G-EDF $O(n)$ ordered list — scheduling overhead rises visibly above ≈ 75 threads; a heap would give $O(\log n)$ (acknowledged, not implemented). P-EDF $O(n/m)$ per core — scales with m .
- **Memory/locking.** G-EDF: one global queue (size n) + one global lock. P-EDF: m queues (size n/m) + m locks — same total memory, less contention per core.
- **Test pessimism at scale.** At 100 processors with light bimodal utilisation, the G-EDF schedulability ratio drops to 0 at cap ≈ 45 — $\approx 55\%$ of capacity wasted by test bounds, not real infeasibility.
- **Period effect.** Shorter periods $\Rightarrow$ higher overhead-to-period ratio $\Rightarrow$ larger schedulability gap (a key reason the OS substrate matters most in short-period regimes).

VI. RELEVANCE FOR EMBEDDED / REAL-TIME SYSTEMS

- Targets the concrete automotive trend of ECU consolidation onto multicore; shows the *OS substrate*, not only the algorithm, decides whether deadlines hold.

- Determinism property with direct engineering value: EPOS tick overhead is thread-count-independent — absent from all Linux-based systems.
- Practical consequence: RTOS choice and WCET budgeting must include real run-queue, timer, IRQ, and cache costs; a low-overhead RTOS narrows the theory–practice schedulability gap.

VII. OPEN POINTS AND QUESTIONS FOR FEEDBACK

- 1) Are G-EDF tests *fundamentally* more pessimistic than P-EDF tests, or is this a maturity gap? (ratio $\rightarrow 0$ at cap 45 even at 100 processors.)
- 2) Can *sampled* “observed worst-case” OS overhead legitimately enter an HRT schedulability proof, or does it weaken the guarantee vs analytical WCET?
- 3) IPI is charged as computation, not self-suspension. How conservative is this, and how much of the theory $\leftrightarrow$ EPOS gap does it explain?
- 4) Is *wbinvd*-forced full eviction a realistic CPMD worst case for periodic control / sensor-fusion tasks?
- 5) Would replacing EPOS’s $O(n)$ list with a heap move the G-EDF/P-EDF crossover into medium utilisation?
- 6) (**Transfer framing**, $\rightarrow$ **M3**) If OS overhead dominates, is the effect observable on general-purpose OSes (Windows/Ubuntu) as a qualitative analogue, and can the resulting deadline miss be proven formally in UPPAAL?